



IT Learning Academy

Filière : Développement Digital, Option Web Full Stack

PROJET DE FIN DE FORMATION

Conception et développement d'un système web de gestion hôtelière

Réalisé par : CHAFIK Abdelbasset

Encadré par : Jawad

Année universitaire : 2025/2026

Dédicace

Je dédie ce travail à mes parents, pour leur soutien, leurs sacrifices et leurs encouragements constants. Je le dédie également à ma famille, à mes proches et à toutes les personnes qui m'ont accompagné durant mon parcours de formation.

À mes formateurs et à mes camarades, je témoigne toute ma reconnaissance pour les échanges, les conseils et l'esprit de collaboration qui ont contribué à l'aboutissement de ce projet.

Remerciements

Je tiens à exprimer ma sincère gratitude à IT Learning Academy pour la qualité de la formation en Développement Digital, Option Web Full Stack, ainsi que pour les moyens pédagogiques mis à disposition tout au long de l'année universitaire.

J'adresse mes remerciements à mon encadrant, Jawad, pour son accompagnement, ses orientations et ses remarques constructives durant la réalisation de ce projet de fin de formation.

Je remercie également l'ensemble des formateurs, mes camarades et mes proches pour leurs encouragements, leur disponibilité et leur soutien. Leur contribution a constitué une aide précieuse dans les différentes étapes d'analyse, de conception, de développement et de validation du système.

Résumé

Ce projet de fin de formation porte sur la conception et le développement d'une application web de gestion hôtelière destinée au personnel d'un établissement. L'objectif principal consiste à centraliser les opérations liées aux utilisateurs, aux clients, aux chambres, aux tarifs, aux réservations, aux paiements, aux réclamations et aux équipements dans une interface unique, sécurisée et responsive.

La solution repose sur une architecture séparant une interface monopage développée avec React et Vite d'une API REST réalisée avec Laravel 11 et connectée à une base de données MySQL. Laravel Sanctum assure l'authentification par jeton. Deux rôles sont distingués : Administrateur et Employé. L'administrateur gère les comptes internes, tandis que chaque utilisateur peut consulter et modifier son propre profil.

Le système calcule la disponibilité réelle des chambres en tenant compte des réservations qui se chevauchent et des maintenances actives. Il prend en charge les réservations multi-chambres, l'occupation, les repas, les périodes tarifaires, les réductions et les politiques de paiement. Les paiements sont tracés par utilisateur, peuvent être annulés sans effacer l'historique et donnent lieu à des reçus et récapitulatifs imprimables ou exportables en PDF.

Une attention particulière a été accordée à la validation des données, à l'affichage cohérent des erreurs, à l'autorisation côté serveur, à la traçabilité et à l'ergonomie. La validation finale a inclus une suite de tests backend ainsi qu'une compilation de production du frontend.

Mots-clés : gestion hôtelière, Laravel, React, Vite, MySQL, réservation, authentification, paiement, API REST.

Abstract

This final training project focuses on the design and development of a web-based hotel management application for internal staff. Its main purpose is to centralize user accounts, customers, rooms, pricing, reservations, payments, complaints and equipment in a secure and responsive interface.

The solution uses a React and Vite single-page application connected to a Laravel 11 REST API and a MySQL database. Laravel Sanctum provides token-based authentication, while administrator and staff roles control access to the system. The application also supports multi-room reservations, real-time room availability rules, pricing periods, meals, discounts, payment policies, audit-friendly payment records and printable PDF documents.

Keywords: hotel management, Laravel, React, Vite, MySQL, reservation, authentication, payment, REST API.

Liste des acronymes

Acronyme	Signification
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
MVC	Model-View-Controller
ORM	Object-Relational Mapping
REST	Representational State Transfer
SPA	Single-Page Application
UML	Unified Modeling Language
UX	User Experience

Table des matières

Dédicace	1
Remerciements	1
Résumé	1
Abstract	1
Liste des acronymes	2
Table des matières	3
Liste des figures	5
Introduction générale	6
CONTEXTE, OBJECTIFS ET CAHIER DES CHARGES	7
1.1 Cadre académique du projet	8
1.2 Contexte général	8
1.3 Problématique	8
1.4 Objectifs	8
1.5 Besoins fonctionnels	8
1.6 Besoins non fonctionnels	9
1.7 Périmètre retenu	9
ANALYSE ET CONCEPTION	10
2.1 Démarche de réalisation	11
2.2 Acteurs du système	11
2.3 Principales règles métier	12
2.4 Architecture générale	14
2.5 Conception des données	14
2.6 Flux d'authentification	17
2.7 Flux de réservation	18
2.8 Diagrammes de séquence	18
RÉALISATION TECHNIQUE	22
3.1 Environnement et technologies	23
3.2 Organisation du frontend	23
3.3 Organisation du backend	23
3.4 Sécurité et autorisation	24
3.5 Validation et gestion des erreurs	24
3.6 Génération de documents et exports	24
3.7 Tests et validation	24
4.1 Connexion et navigation	26
4.2 Profil utilisateur	26
4.3 Administration des utilisateurs	26

4.4 Tableau de bord	26
4.5 Gestion des clients.....	27
4.5.1 Clients particuliers	27
4.5.2 Clients sociétés.....	27
4.6 Gestion des chambres.....	27
4.6.1 État des chambres	27
4.7 Gestion des tarifs	28
4.7.1 Périodes tarifaires	28
4.7.2 Tarifs des chambres.....	28
4.7.3 Tarifs des repas.....	28
4.7.4 Réductions	28
4.8 Gestion des réservations	28
4.9 Paiements et documents	29
4.10 Réclamations	29
4.11 Équipements	30
4.12 Recherche, filtres, exports et validation.....	30
5.1 Résultats obtenus	32
5.2 Difficultés rencontrées et solutions.....	32
5.3 Limites actuelles	32
5.4 Perspectives.....	33
Conclusion générale.....	33
Bibliographie et webographie	33
Annexes	33
Annexe A — Principaux endpoints.....	33
Annexe B — Captures et diagrammes à fournir	34

Liste des figures

Figure 1 — Diagramme de cas d'utilisation	12
Figure 2 — Architecture générale de la solution	14
Figure 3 — Diagramme de classes / domaine	15
Figure 4 — Modèle relationnel de la base de données	16
Figure 5 — Flux d'authentification et d'autorisation	17
Figure 6 — Workflow principal de création d'une réservation.....	18
Figure 7 — Séquence : création d'une réservation	19
Figure 8 — Séquence : enregistrement et annulation d'un paiement.....	20
Figure 9 — Séquence : traitement d'une réclamation	21
Figure 10 — Page de connexion	26
Figure 11 — Navigation et identité de l'utilisateur.....	26
Figure 12 — Page Mon profil.....	26
Figure 13 — Liste et filtres des utilisateurs	26
Figure 14 — Création ou modification d'un utilisateur	26
Figure 15 — Tableau de bord et statistiques.....	26
Figure 16 — Liste des clients particuliers.....	27
Figure 17 — Formulaire client particulier et validation.....	27
Figure 18 — Liste des clients sociétés.....	27
Figure 19 — Formulaire client société et contacts.....	27
Figure 20 — Gestion des chambres	27
Figure 21 — Gestion des types de chambres.....	27
Figure 22 — État des chambres.....	28
Figure 23 — Périodes tarifaires	28
Figure 24 — Tarifs des chambres.....	28
Figure 25 — Tarifs des repas.....	28
Figure 26 — Plans et types de réductions.....	28
Figure 27 — Liste des réservations	29
Figure 28 — Sélection du client et des dates	29
Figure 29 — Affectation multi-chambres et occupation.....	29
Figure 30 — Repas, politique de paiement et calcul.....	29
Figure 31 — Historique des paiements et audit.....	29
Figure 32 — Reçu de paiement.....	29
Figure 33 — Récapitulatif de règlement.....	29
Figure 34 — Liste et filtres des réclamations.....	30
Figure 35 — Détail et workflow de traitement.....	30
Figure 36 — Liste des équipements	30
Figure 37 — Formulaire d'équipement et localisation conditionnelle.....	30
Figure 38 — Exemple de validation inline et champs obligatoires.....	30

Introduction générale

La transformation numérique des activités hôtelières nécessite des outils capables de centraliser un volume important d'informations tout en limitant les erreurs de saisie et les opérations manuelles. Dans un établissement, la disponibilité des chambres, les données des clients, les tarifs, les paiements, les réclamations et la maintenance doivent rester cohérents afin de permettre au personnel de prendre des décisions rapidement.

Le présent projet de fin de formation consiste à concevoir et à développer un système web de gestion hôtelière destiné à un usage interne. L'application a été pensée comme une plateforme centralisée permettant à des utilisateurs authentifiés de gérer l'ensemble des opérations courantes. Elle distingue les responsabilités d'un administrateur de celles d'un employé et applique des règles métier afin de protéger l'intégrité des données.

La réalisation a mobilisé Laravel 11 pour l'API et la logique métier, React avec Vite pour l'interface utilisateur et MySQL pour la persistance des données. Le projet s'appuie également sur une validation serveur, une gestion cohérente des erreurs, des exports, des documents PDF et une suite de tests automatisés.

Ce rapport présente d'abord le contexte, les objectifs et les besoins fonctionnels du système. Il expose ensuite l'analyse, la conception, l'architecture et les principales règles métier. Les chapitres suivants détaillent la réalisation technique, les fonctionnalités obtenues, les difficultés rencontrées, les résultats, les limites et les perspectives d'amélioration.

CHAPITRE 1

CONTEXTE, OBJECTIFS ET CAHIER DES CHARGES



1.1 Cadre académique du projet

Ce système a été réalisé dans le cadre du projet de fin de formation de la filière Développement Digital, Option Web Full Stack, à IT Learning Academy durant l'année universitaire 2025/2026. Il constitue une mise en application des compétences acquises en analyse, modélisation, développement frontend, développement backend, bases de données, sécurité, tests et gestion de versions.

1.2 Contexte général

La gestion d'un hôtel implique plusieurs domaines étroitement liés. Une réservation dépend de la disponibilité d'une chambre, de son état de maintenance, du type de client, de la période tarifaire et des services sélectionnés. Les paiements doivent rester associés à la réservation et à l'utilisateur qui les a saisis. Les réclamations et les équipements nécessitent, eux aussi, un suivi précis.

Lorsque ces informations sont dispersées ou gérées manuellement, les risques de double réservation, d'erreur tarifaire, de perte d'historique et de retard de traitement augmentent. Le projet répond à ce besoin par une plateforme web centralisée, destinée au personnel interne et accessible uniquement après authentification.

1.3 Problématique

Problématique centrale

Comment concevoir une application web sécurisée, maintenable et ergonomique capable de centraliser les opérations hôtelières, d'appliquer les règles de disponibilité et de tarification, et de préserver une traçabilité fiable des actions ?

1.4 Objectifs

- **Centraliser les données** : regrouper dans une même application les utilisateurs, les clients, les chambres, les tarifs, les réservations, les paiements, les réclamations et les équipements.
- **Sécuriser l'accès** : limiter l'application aux comptes internes actifs et protéger l'administration des utilisateurs par un rôle spécifique.
- **Automatiser les règles métier** : empêcher les conflits de réservation, calculer les tarifs applicables et préserver les historiques.
- **Améliorer l'expérience utilisateur** : fournir une interface responsive, des formulaires cohérents, des erreurs compréhensibles et des outils de recherche, de filtrage et d'export.
- **Assurer la qualité** : valider les données côté serveur, utiliser des transactions lorsque nécessaire et vérifier le comportement à l'aide de tests automatisés et manuels.

1.5 Besoins fonctionnels

Module	Fonctions principales
Authentification et profils	Connexion sécurisée, compte actif, déconnexion, consultation et modification du profil, changement de mot de passe et photo.
Administration des utilisateurs	Création de comptes, rôles Administrateur/Employé, activation ou désactivation, modification et réinitialisation de mot de passe.

Module	Fonctions principales
Clients	Gestion distincte des clients particuliers et sociétés, contacts, enfants, localisation et paramètres de crédit.
Chambres	Gestion des chambres physiques, types, capacités, étages, vues, propreté et maintenance.
Tarifs	Plans de tarifs chambre, repas et réduction, ainsi que périodes tarifaires applicables.
Réservations	Sélection du client et des dates, recherche de disponibilité, affectation multi-chambres, occupation, repas, calcul et statut.
Paiements	Saisie, annulation, audit utilisateur, historique, reçu et récapitulatif imprimable ou PDF.
Réclamations	Création, priorité, traitement, réponse, résolution et annulation.
Équipements	Catégories, localisation, état, acquisition, garantie, fournisseur, prix et documents.

1.6 Besoins non fonctionnels

- **Sécurité** : mots de passe hachés, jetons Sanctum, protection des routes, contrôle du rôle et du statut du compte.
- **Intégrité** : validation Laravel, contraintes relationnelles, transactions et règles empêchant les états incohérents.
- **Ergonomie** : navigation homogène, formulaires responsive, champs obligatoires identifiables et erreurs affichées sous les contrôles concernés.
- **Performance** : chargement paginé des listes, filtrage, réutilisation de composants et compilation optimisée avec Vite.
- **Maintenabilité** : séparation frontend/backend, composants partagés, hooks, services, Form Requests et tests.
- **Traçabilité** : conservation de l'auteur des paiements et des annulations, même lorsque le compte est désactivé.

1.7 Périmètre retenu

Le périmètre couvre une application interne destinée au personnel de l'hôtel. Il n'inclut pas de portail public permettant au client final de réserver lui-même, ni de facturation fiscale complète, ni d'intégration bancaire réelle. Les documents de paiement générés sont des reçus et des récapitulatifs opérationnels, et non des factures légales.

CHAPITRE 2

ANALYSE ET CONCEPTION



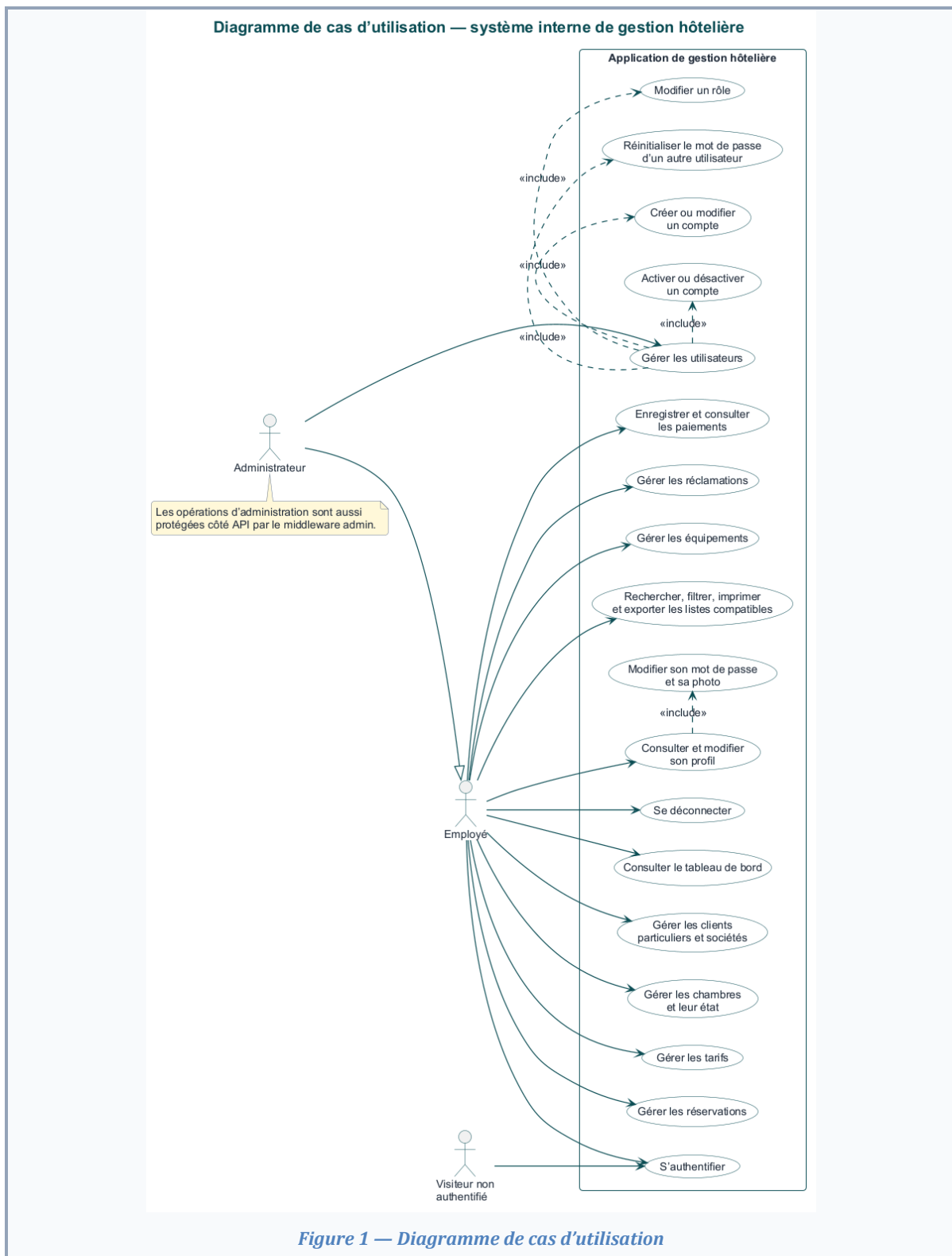
2.1 Démarche de réalisation

La réalisation a suivi une démarche itérative. Chaque module a été analysé, implémenté, testé puis amélioré progressivement. Les corrections ont été organisées par lots fonctionnels afin de limiter les régressions : authentification, interface globale, clients, disponibilités, paiements, utilisateurs, puis validation des formulaires.

Git et GitHub ont permis de conserver l'historique des modifications. Les évolutions importantes ont été vérifiées à l'aide de tests backend, de compilations frontend et de scénarios manuels dans le navigateur.

2.2 Acteurs du système

Acteur	Responsabilités
Administrateur	Accède à tous les modules opérationnels et gère les comptes internes, les rôles, les statuts et les réinitialisations de mot de passe.
Employé	Utilise les fonctions opérationnelles de l'hôtel et gère son propre profil, sans pouvoir administrer les autres comptes.



2.3 Principales règles métier

Règle métier	Comportement attendu
Compte utilisateur	Un compte inactif ne peut ni obtenir un nouveau jeton ni utiliser un ancien jeton.
Administration	Seul un administrateur peut gérer les utilisateurs. Le dernier administrateur actif ne peut pas être désactivé ou

Règle métier	Comportement attendu
	rétrogradé.
Disponibilité	Une chambre est exclue si une réservation active chevauche le séjour ou si une maintenance de la chambre est active.
Sélection multi-chambres	Une chambre choisie dans une ligne n'est plus proposée dans les autres lignes de la même réservation.
Tarification	La période applicable relie les plans chambre, repas et réduction utilisés pour le calcul.
Paielement	Chaque paiement est associé à l'utilisateur authentifié. Une annulation ne supprime pas l'historique.
Réclamation	Une réponse existante est obligatoire avant de marquer une réclamation comme résolue.
Codes	Les codes clients et tarifs de chambre sont générés côté backend et restent immuables.

2.4 Architecture générale

L'application adopte une architecture en couches. Le frontend React gère l'interface et l'état d'affichage. Les requêtes HTTP sont envoyées à l'API Laravel avec Axios. Laravel applique l'authentification, l'autorisation, la validation et les règles métier avant d'accéder aux données MySQL à travers Eloquent.

Architecture générale de la solution

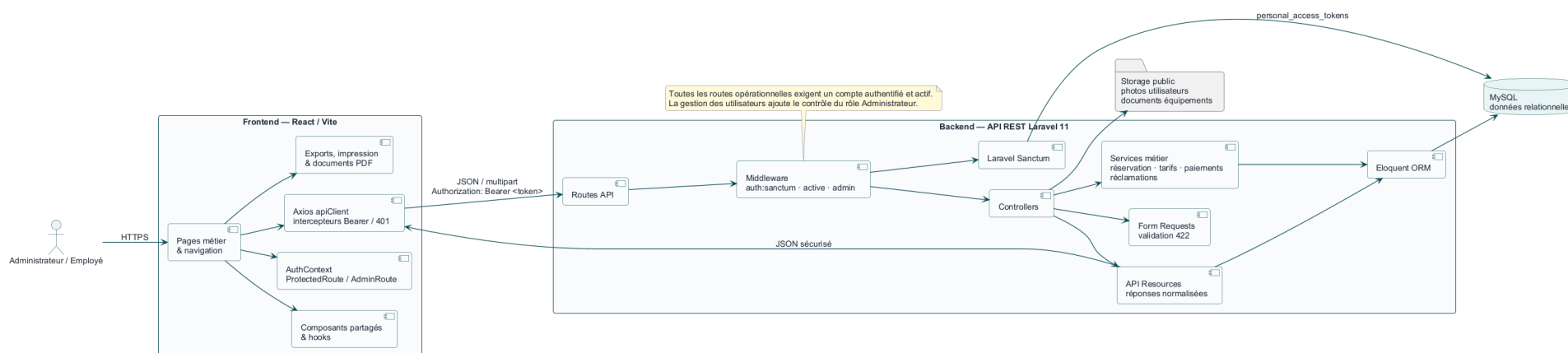
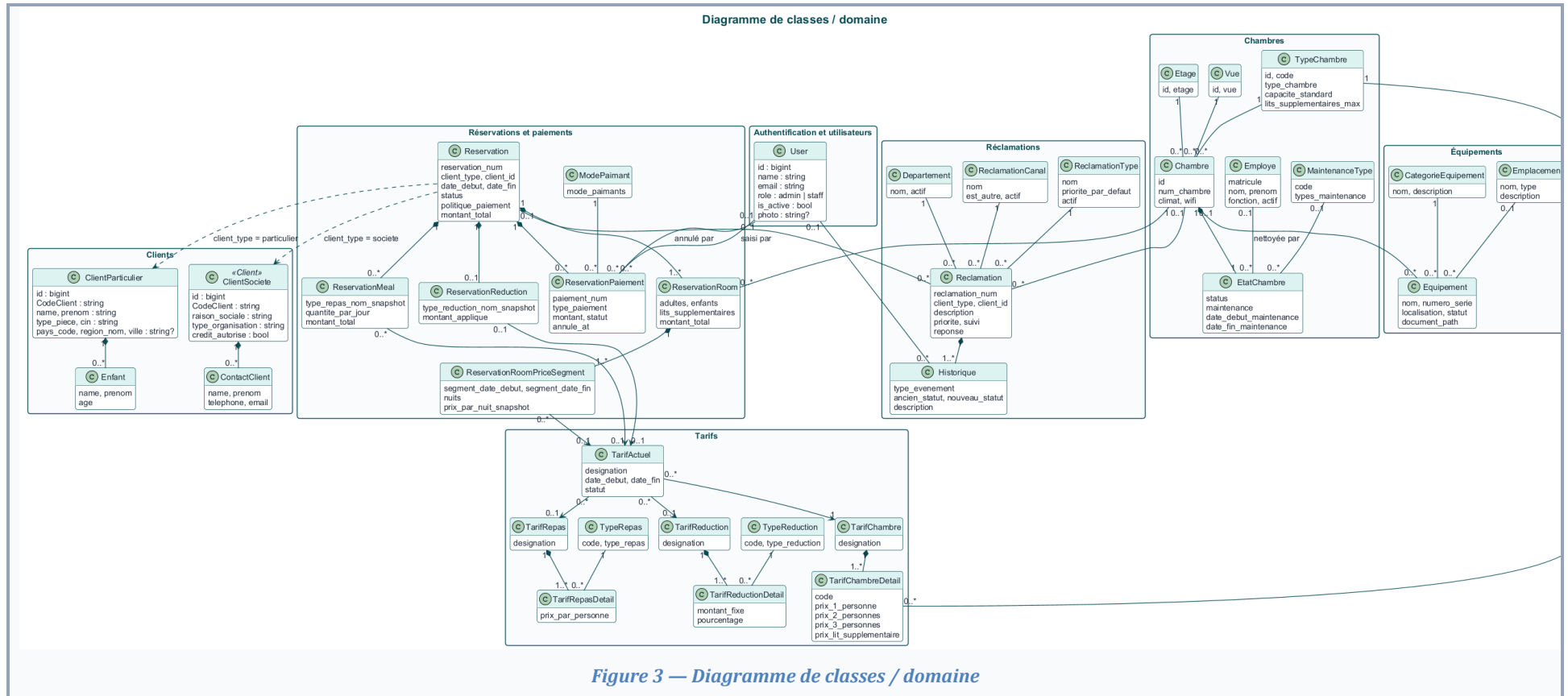


Figure 2 — Architecture générale de la solution

2.5 Conception des données

Le modèle de données est organisé autour de plusieurs domaines : utilisateurs, clients, chambres, tarifs, réservations, paiements, réclamations et équipements. Les relations sont représentées par des clés étrangères et chargées par Eloquent selon les besoins de l'API.

- **Utilisateurs** : rôle, statut actif, profil, jetons Sanctum, création et annulation des paiements.
- **Clients** : client particulier avec enfants éventuels, client société avec contacts et paramètres de crédit.
- **Chambres** : chambre physique, type, capacité, étage, vue, état de propreté et maintenance.
- **Tarifs** : plans et détails de prix, types de repas, réductions et périodes tarifaires.
- **Réservations** : client, dates, statut, chambres affectées, occupation, repas, politique de paiement et calculs.
- **Paiements** : montant, moyen, référence, créateur, annulation, annuleur et motif.
- **Réclamations et équipements** : suivi opérationnel, statuts, priorités, catégories et localisations.



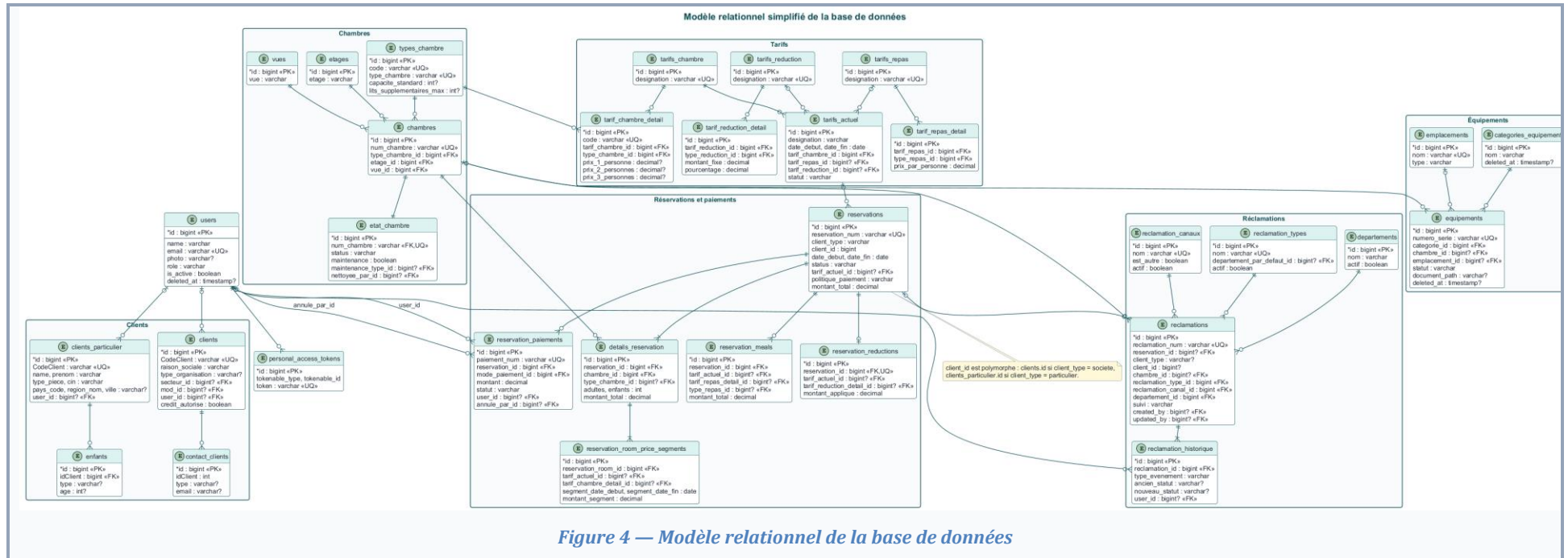


Figure 4 — Modèle relationnel de la base de données

2.6 Flux d'authentification

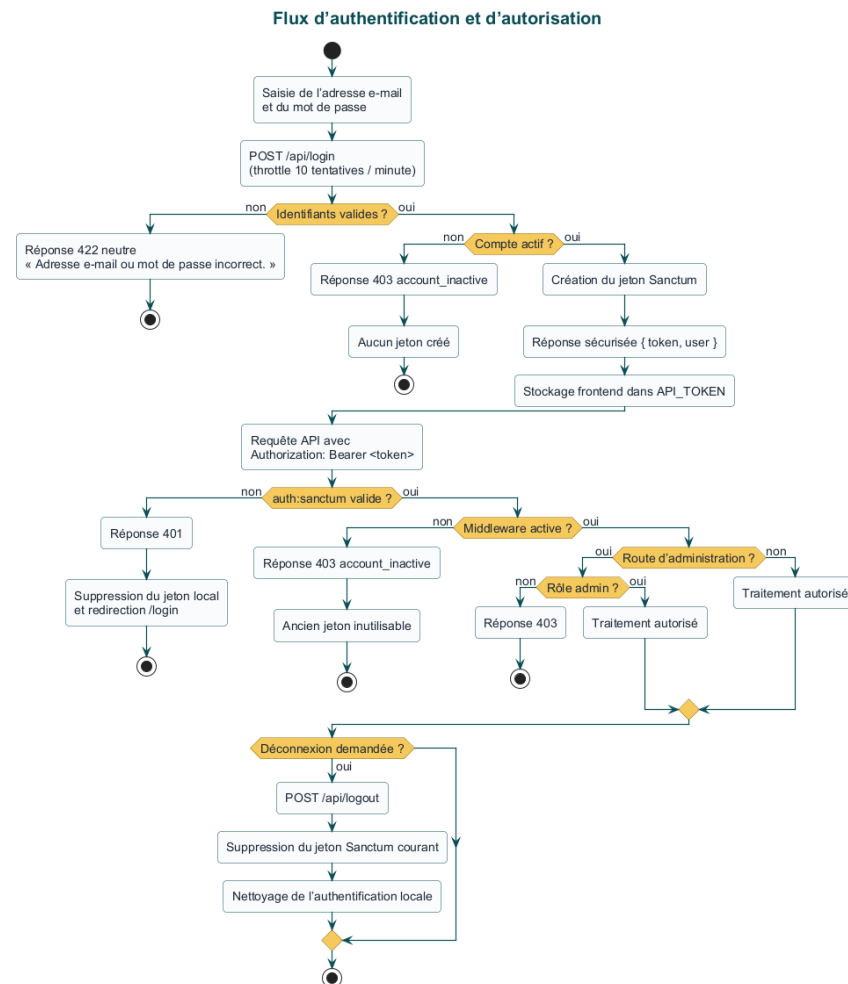


Figure 5 — Flux d'authentification et d'autorisation

2.7 Flux de réservation

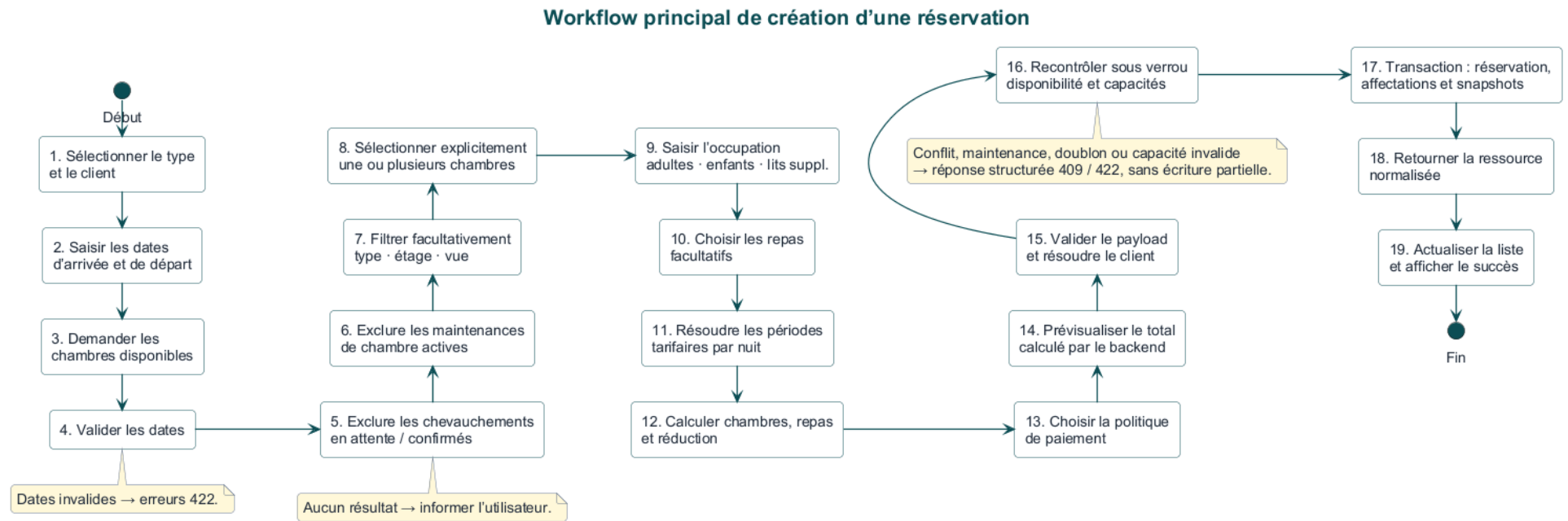


Figure 6 — Workflow principal de création d'une réservation

2.8 Diagrammes de séquence

Les diagrammes de séquence suivants décrivent les échanges réellement implémentés entre l'utilisateur, le frontend React, l'API Laravel et la base de données. Ils couvrent la création d'une réservation, l'enregistrement et l'annulation d'un paiement, ainsi que le traitement d'une réclamation.

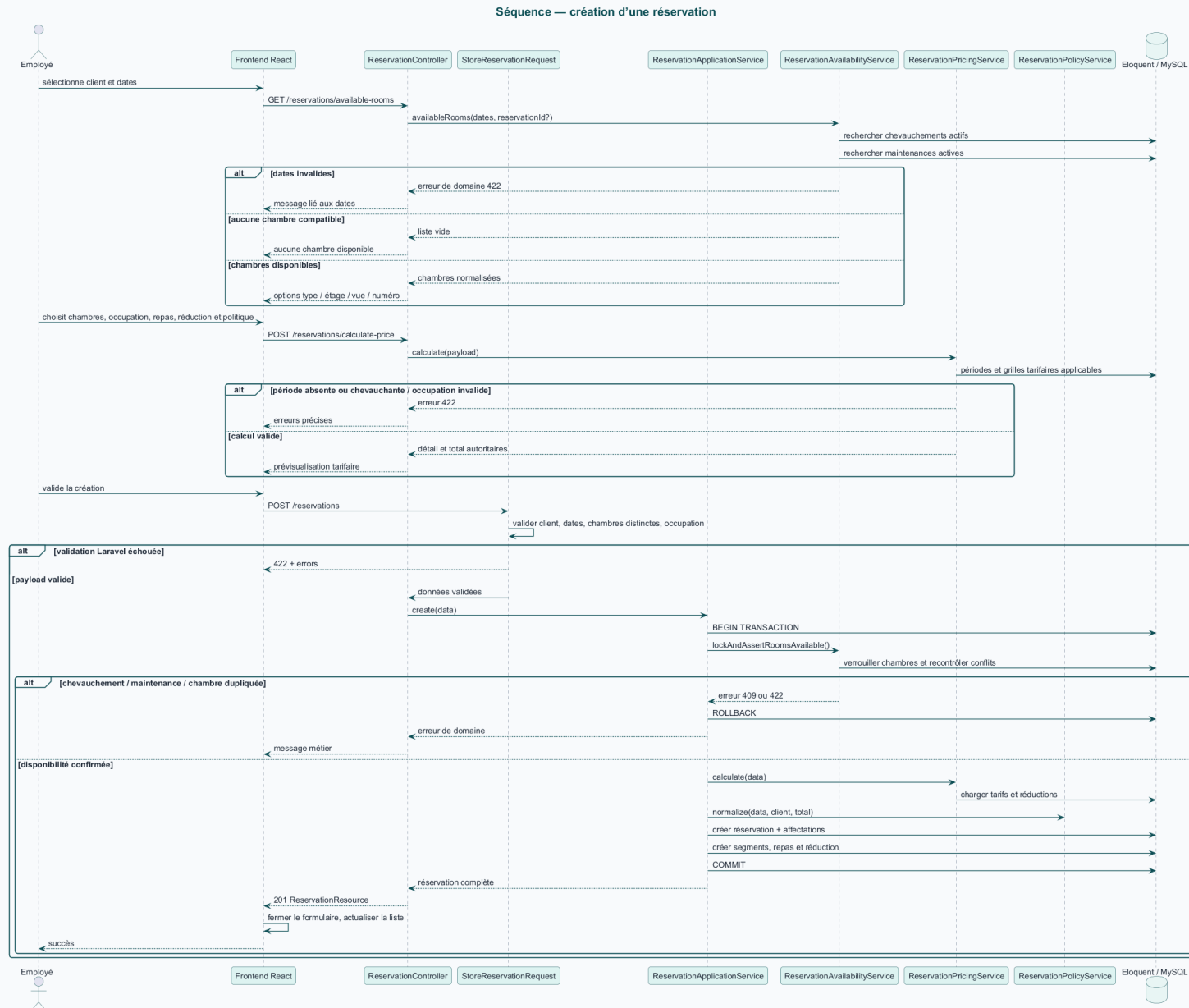


Figure 7 — Séquence : création d'une réservation

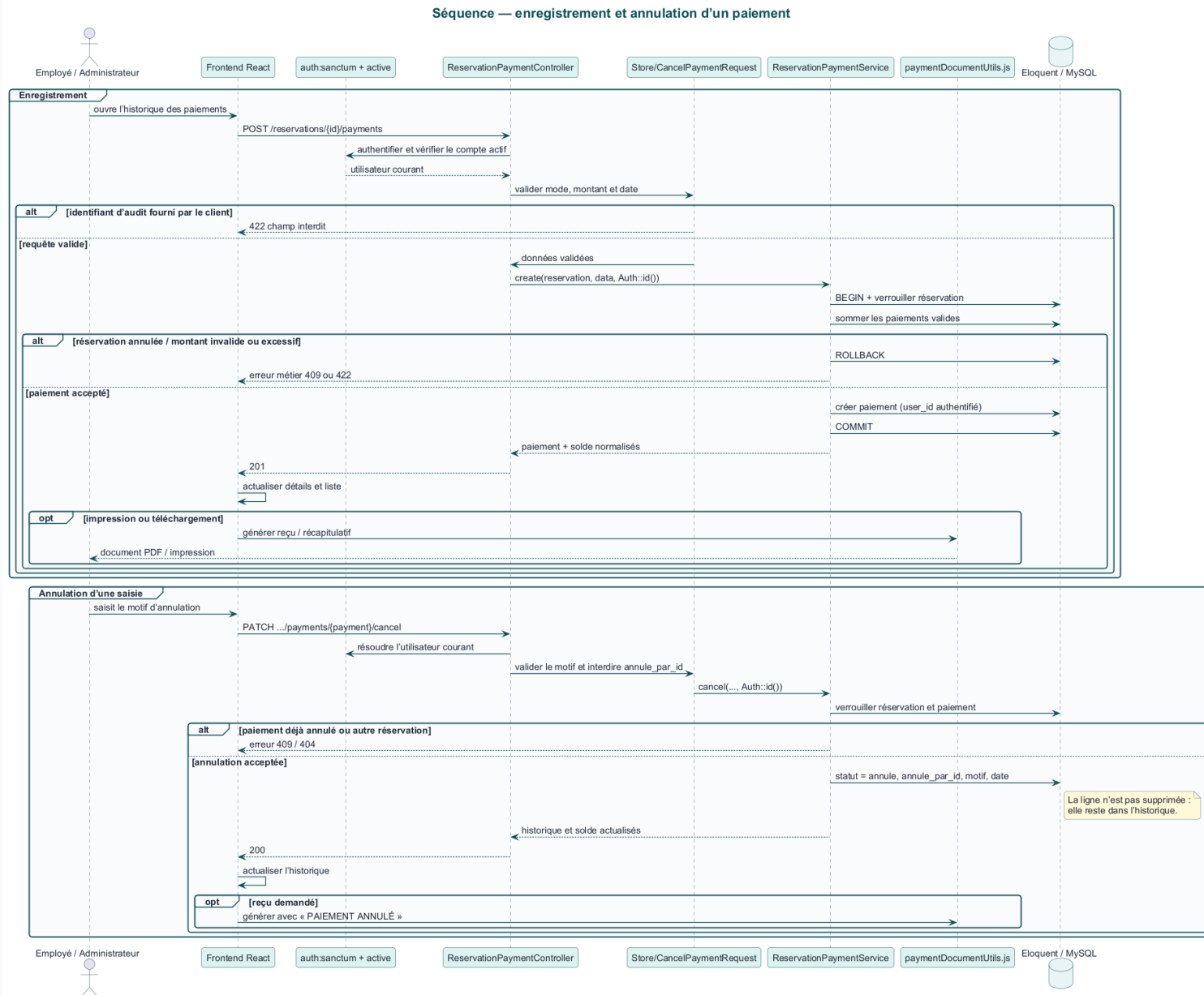


Figure 8 — Séquence : enregistrement et annulation d'un paiement

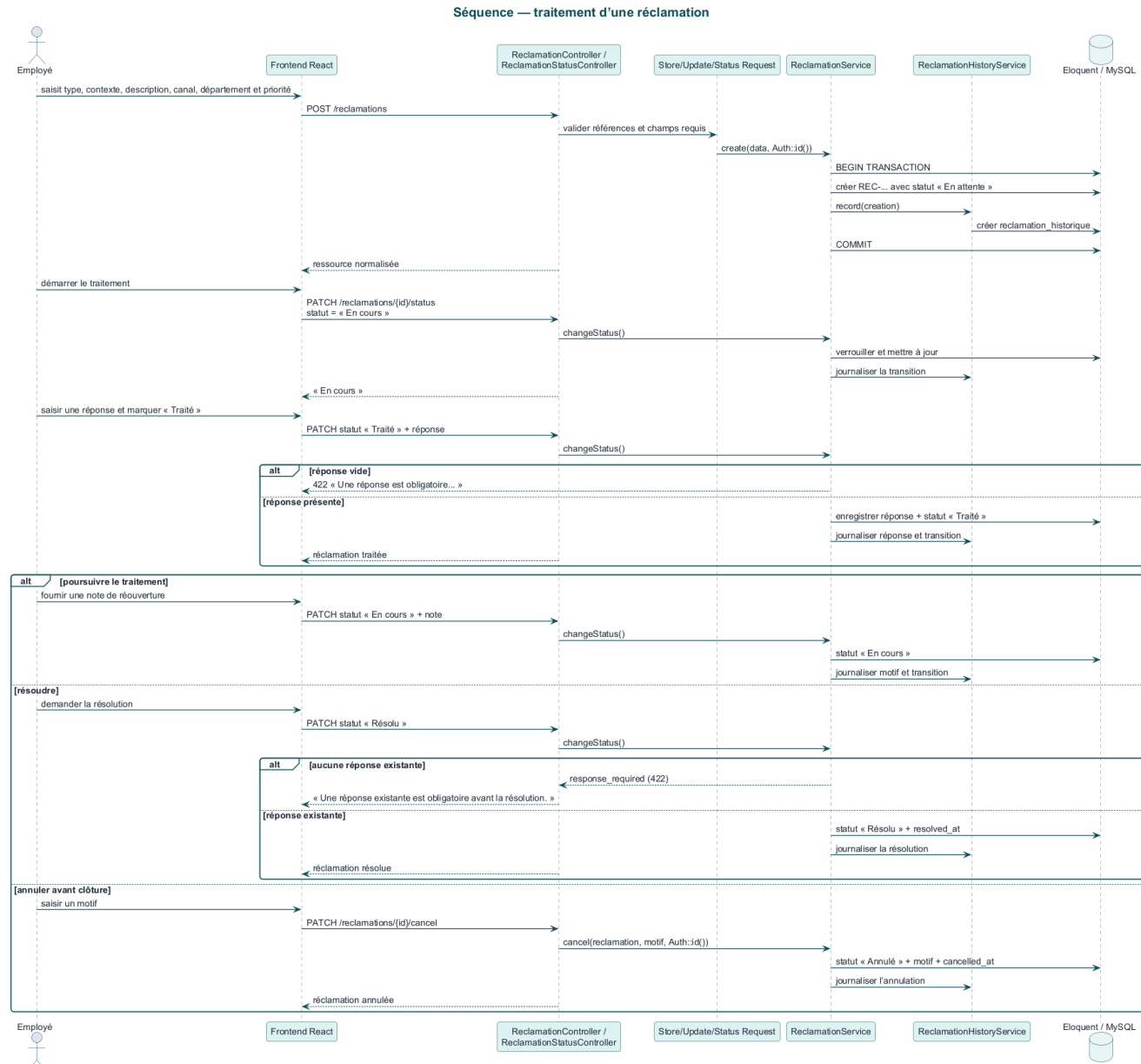


Figure 9 — Séquence : traitement d'une réclamation

CHAPITRE 3

RÉALISATION TECHNIQUE



3.1 Environnement et technologies

Technologie / outil	Rôle dans le projet	Apport principal
Laravel 11	API REST, règles métier, Eloquent, migrations et validation	Structure backend robuste et sécurisée
Laravel Sanctum	Authentification par bearer token	Protection des routes et des sessions
React	Interface monopage à base de composants	Réactivité et réutilisation
Vite	Serveur de développement et build	Compilation rapide et production optimisée
Axios	Communication HTTP avec l'API	Intercepteurs, jeton et gestion des erreurs
MySQL	Base de données relationnelle	Intégrité et relations entre domaines
React-Bootstrap / Bootstrap	Formulaires, modales et composants responsive	Cohérence visuelle
Material UI	Composants et mise en page complémentaires	Ergonomie de certaines interfaces
SweetAlert2	Confirmations et notifications opérationnelles	Retour utilisateur clair
jsPDF / AutoTable	Exports PDF et documents de paiement	Reçus, récapitulatifs et listes
PHPUnit	Tests fonctionnels backend	Vérification automatisée
Git / GitHub	Gestion de versions	Historique et collaboration
Postman	Vérification manuelle de l'API	Tests rapides des endpoints

3.2 Organisation du frontend

Le frontend est structuré en pages, composants partagés, hooks et utilitaires. Le contexte d'authentification conserve l'utilisateur connecté et permet de rafraîchir son profil. Les routes protégées empêchent l'accès aux écrans internes sans jeton, tandis qu'un garde spécifique limite la page des utilisateurs au rôle administrateur.

- **Composants partagés** : navigation, statistiques, recherche, pagination, état des listes, export, avatar et labels obligatoires.
- **Hooks** : gestion des formulaires complexes, notamment la réservation et les contrôles de listes.
- **Utilitaires** : normalisation des recherches, génération de documents, formatage et gestion cohérente des erreurs de validation.
- **Styles** : thème global, tableaux à défilement local, drawers, modales et adaptation mobile.

3.3 Organisation du backend

Le backend suit les conventions Laravel. Les contrôleurs reçoivent les requêtes, les Form Requests valident les données, les modèles Eloquent représentent les entités et les services isolent certaines règles complexes. Les migrations assurent l'évolution de la base de données.

- **Authentification** : contrôleur de connexion, Sanctum, middleware de compte actif et middleware administrateur.
- **Validation** : règles centralisées, messages compréhensibles et réponses HTTP 422 structurées.

- **Transactions** : utilisées pour les opérations qui doivent rester atomiques, comme les changements touchant le dernier administrateur.
- **Ressources API** : limitation des champs exposés afin de ne jamais retourner les mots de passe, jetons ou informations sensibles.

3.4 Sécurité et autorisation

La sécurité ne repose pas uniquement sur l'interface. Toutes les routes opérationnelles sont protégées par l'authentification Sanctum et par un contrôle du statut actif. Les endpoints d'administration ajoutent une vérification du rôle. La dissimulation d'un lien dans la navigation n'est donc jamais la seule protection.

- Absence de route publique d'inscription.
- Limitation des tentatives de connexion.
- Hachage automatique des mots de passe.
- Révocation des jetons lors de la désactivation ou de la réinitialisation administrative du mot de passe.
- Préservation du jeton courant lors du changement personnel de mot de passe, avec révocation des autres sessions.
- Protection contre l'auto-désactivation, l'auto-rétrogradation et la suppression du dernier administrateur actif.

3.5 Validation et gestion des erreurs

Les formulaires utilisent une couche partagée de validation. Les champs obligatoires portent un astérisque visible avant la soumission. Lorsqu'une erreur survient, le contrôle reçoit un état rouge et le message exact est affiché directement sous le champ. Les erreurs Laravel 422 sont normalisées, y compris pour les chemins imbriqués des contacts, enfants, chambres, repas et tarifs.

Les erreurs de champ ne déclenchent plus de fenêtre SweetAlert. Les fenêtres restent réservées aux confirmations, aux succès et aux erreurs générales qui ne peuvent pas être associées à un contrôle précis. Le premier champ invalide visible reçoit le focus ou est amené dans la zone d'affichage.

3.6 Génération de documents et exports

Les listes peuvent être exportées au format Excel ou PDF et imprimées. Pour les paiements, l'application génère un récapitulatif de règlement et un reçu individuel. Les documents indiquent l'utilisateur ayant saisi le paiement et, en cas d'annulation, le statut et les informations d'annulation. La terminologie de facture n'est pas utilisée puisque les informations fiscales complètes ne font pas partie du périmètre.

3.7 Tests et validation

La qualité du backend a été vérifiée par des tests fonctionnels couvrant l'authentification, les autorisations, les comptes utilisateurs, les réservations, les paiements et les autres modules. Au dernier contrôle global, la suite backend comportait 220 tests et 1 481 assertions réussies. Le frontend a également passé une compilation de production avec succès.

Contrôles réalisés

Tests backend ciblés et complets, vérification syntaxique PHP, audit des routes, compilation frontend, contrôle git diff --check et scénarios manuels authentifiés.

CHAPITRE 4

PRÉSENTATION FONCTIONNELLE DE LA SOLUTION



4.1 Connexion et navigation

La page de connexion constitue le seul accès public. Après authentification, l'utilisateur est redirigé vers l'application. La navigation latérale présente les modules selon le rôle, conserve la déconnexion dans une zone fixe et affiche un bloc d'identité donnant accès au profil.

Figure 10 — Page de connexion

Capture de la version finale à insérer

Figure 11 — Navigation et identité de l'utilisateur

Capture de la version finale à insérer

4.2 Profil utilisateur

Chaque utilisateur peut modifier son nom, son adresse électronique et sa photo. Le rôle et le statut sont affichés en lecture seule. Le changement de mot de passe exige le mot de passe actuel et révoque les autres sessions lorsque la session courante peut être conservée.

Figure 12 — Page Mon profil

Capture de la version finale à insérer

4.3 Administration des utilisateurs

La page Gestion des utilisateurs est réservée aux administrateurs. Elle permet la recherche, le filtrage, la pagination, la création de comptes, la modification des informations et du rôle, l'activation ou la désactivation, ainsi que la réinitialisation du mot de passe d'un autre utilisateur.

La désactivation est préférée à la suppression afin de préserver les historiques. Le système empêche les opérations qui laisseraient l'application sans administrateur actif.

Figure 13 — Liste et filtres des utilisateurs

Capture de la version finale à insérer

Figure 14 — Création ou modification d'un utilisateur

Capture de la version finale à insérer

4.4 Tableau de bord

Le tableau de bord fournit une synthèse de l'activité hôtelière. Les indicateurs portent notamment sur les réservations, les chambres, les réclamations et les équipements. Ils permettent d'identifier rapidement les éléments qui nécessitent une attention opérationnelle.

Figure 15 — Tableau de bord et statistiques

Capture de la version finale à insérer

4.5 Gestion des clients

4.5.1 Clients particuliers

Les clients particuliers disposent d'un code généré automatiquement au format CP-000001. Le formulaire gère l'identité, la pièce d'identité, la nationalité, le téléphone, la localisation et les enfants éventuels. L'âge d'un enfant est facultatif mais, lorsqu'il est renseigné, il doit être compris entre 0 et 17 ans.

Figure 16 — Liste des clients particuliers

Capture de la version finale à insérer

Figure 17 — Formulaire client particulier et validation

Capture de la version finale à insérer

4.5.2 Clients sociétés

Les clients sociétés utilisent un code généré au format CS-000001. Le système gère la raison sociale, l'ICE, le type d'organisation, le secteur, les coordonnées, la localisation, les contacts et les paramètres de crédit. Les contacts sont saisis dans des lignes indépendantes et leurs erreurs restent associées à la bonne ligne.

Figure 18 — Liste des clients sociétés

Capture de la version finale à insérer

Figure 19 — Formulaire client société et contacts

Capture de la version finale à insérer

4.6 Gestion des chambres

Le module Chambres distingue le type de chambre, qui représente un standard, et la chambre physique identifiée par un numéro. Les types regroupent notamment la capacité, le nombre de lits, le nombre de salles et les lits supplémentaires. Chaque chambre est liée à un étage et à une vue, et peut disposer de la climatisation et du Wi-Fi.

Figure 20 — Gestion des chambres

Capture de la version finale à insérer

Figure 21 — Gestion des types de chambres

Capture de la version finale à insérer

4.6.1 État des chambres

La page État des chambres suit la propreté et la maintenance. La propreté utilise les valeurs Nettoyée et Non nettoyée. Une maintenance active rend la chambre indisponible pour les nouvelles réservations sur la période concernée.

Figure 22 — État des chambres

Capture de la version finale à insérer

4.7 Gestion des tarifs

4.7.1 Périodes tarifaires

Une période tarifaire définit les dates d'application et relie un plan de tarifs de chambres, un plan de repas et, lorsque nécessaire, un plan de réductions. Son statut permet de préparer, activer puis archiver une configuration tarifaire.

Figure 23 — Périodes tarifaires

Capture de la version finale à insérer

4.7.2 Tarifs des chambres

Les tarifs de chambres sont associés à un type de chambre et à un plan. Les montants peuvent varier selon le nombre d'occupants et incluent le prix du lit supplémentaire. Le code du tarif est généré automatiquement par le backend.

Figure 24 — Tarifs des chambres

Capture de la version finale à insérer

4.7.3 Tarifs des repas

Les types de repas sont associés à un prix par personne dans un plan tarifaire. Ils peuvent ensuite être sélectionnés lors d'une réservation avec une quantité quotidienne adaptée au nombre d'occupants.

Figure 25 — Tarifs des repas

Capture de la version finale à insérer

4.7.4 Réductions

Les réductions sont exprimées sous forme de montant fixe ou de pourcentage. La validation garantit qu'une valeur utile est renseignée sans imposer à tort les deux champs simultanément.

Figure 26 — Plans et types de réductions

Capture de la version finale à insérer

4.8 Gestion des réservations

La réservation constitue le cœur du système. L'utilisateur sélectionne d'abord le type de client et le client, puis saisit les dates de séjour. L'API retourne uniquement les chambres réellement disponibles, après exclusion des conflits de réservation et des maintenances actives.

Chaque ligne de chambre comporte des filtres facultatifs par type, étage et vue, ainsi qu'un numéro de chambre explicite. Les sélecteurs peuvent être utilisés dans n'importe quel ordre. L'occupation est

enregistrée par chambre et plusieurs chambres peuvent être affectées à la même réservation sans doublon.

Les repas, les réductions et la politique de paiement participent au calcul. Le système prend en charge les statuts En attente, Confirmé et Annulé, ainsi que la consultation, la modification et la suppression selon les règles existantes.

Figure 27 — Liste des réservations

Capture de la version finale à insérer

Figure 28 — Sélection du client et des dates

Capture de la version finale à insérer

Figure 29 — Affectation multi-chambres et occupation

Capture de la version finale à insérer

Figure 30 — Repas, politique de paiement et calcul

Capture de la version finale à insérer

4.9 Paiements et documents

L'historique des règlements est intégré à la réservation. Lors de la création d'un paiement, le backend utilise exclusivement l'utilisateur authentifié pour renseigner Saisi par. Les identifiants d'audit envoyés par le frontend sont rejetés afin d'empêcher toute falsification.

Un paiement peut être annulé sans être supprimé. Le motif, l'utilisateur et la date d'annulation restent accessibles. Les paiements anciens dont l'auteur n'était pas enregistré affichent un tiret plutôt qu'une attribution inventée.

Deux documents sont disponibles : un récapitulatif global de règlement pour la réservation et un reçu par paiement. Les paiements annulés restent imprimables avec la mention PAIEMENT ANNULÉ.

Figure 31 — Historique des paiements et audit

Capture de la version finale à insérer

Figure 32 — Reçu de paiement

Capture de la version finale à insérer

Figure 33 — Récapitulatif de règlement

Capture de la version finale à insérer

4.10 Réclamations

Le module Réclamations permet d'enregistrer la demande d'un client, d'attribuer une priorité et un département, puis de suivre le traitement. Les statuts utilisés sont En attente, En cours, Traité, Résolu et

Annulé. La résolution exige qu'une réponse existe déjà, ce qui empêche de clôturer un dossier sans trace du traitement effectué.

Figure 34 — Liste et filtres des réclamations

Capture de la version finale à insérer

Figure 35 — Détail et workflow de traitement

Capture de la version finale à insérer

4.11 Équipements

La gestion des équipements couvre le nom, le numéro de série, la marque, le modèle, la catégorie, le statut, la date d'acquisition, la garantie, le fournisseur, le prix d'achat et un document éventuel. Un équipement peut être affecté soit à une chambre, soit à un emplacement interne.

Les statuts sont présentés sous les libellés En service, En maintenance et Hors service. La garantie est mise en évidence selon qu'elle est valide, proche de l'expiration, expirée ou non renseignée.

Figure 36 — Liste des équipements

Capture de la version finale à insérer

Figure 37 — Formulaire d'équipement et localisation conditionnelle

Capture de la version finale à insérer

4.12 Recherche, filtres, exports et validation

Les principales listes proposent une recherche, des filtres, une pagination et, lorsque cela est pertinent, des exports Excel/PDF et une impression. Les termes recherchés sont mis en évidence dans les tableaux. Les formulaires utilisent le même style d'erreur dans les différents modules, ce qui réduit les incohérences d'interface.

Figure 38 — Exemple de validation inline et champs obligatoires

Capture de la version finale à insérer

CHAPITRE 5

RÉSULTATS, DIFFICULTÉS ET PERSPECTIVES



5.1 Résultats obtenus

Le projet aboutit à une application web fonctionnelle couvrant l'ensemble du périmètre prévu. Les modules partagent une navigation cohérente et des composants communs. Les règles critiques sont appliquées côté serveur, tandis que l'interface fournit des retours clairs et conserve les valeurs saisies lors d'une erreur de validation.

- Authentification interne sécurisée et gestion des rôles.
- Administration des utilisateurs et profil personnel.
- Gestion structurée des clients, chambres, tarifs, réservations et services.
- Disponibilité calculée en fonction des réservations et de la maintenance.
- Paiements traçables avec documents imprimables et PDF.
- Workflow de réclamation avec règle de résolution.
- Gestion des équipements et de leur garantie.
- Tests backend réussis et build frontend validé.

5.2 Difficultés rencontrées et solutions

Difficulté	Solution retenue
Disponibilité des chambres	La combinaison des dates, réservations existantes, maintenances et sélections multi-lignes nécessitait une logique centralisée. La solution a consisté à isoler le calcul et à maintenir des filtres indépendants dans chaque ligne.
Cohérence de l'interface	Les pages historiques utilisaient des comportements différents. Un thème commun, des composants partagés et une couche de validation uniforme ont été introduits.
Traçabilité des paiements	Les anciens paiements ne contenaient pas toujours d'auteur. Les champs existants ont été réutilisés sans fabriquer d'historique, et toutes les nouvelles opérations utilisent l'utilisateur authentifié.
Gestion des comptes	La désactivation ne devait pas supprimer l'historique. Des rôles simples, un statut actif, la révocation des jetons et la protection du dernier administrateur ont été mis en place.
Formulaires complexes	Les erreurs imbriquées de contacts, chambres et repas étaient difficiles à associer au bon contrôle. Des utilitaires communs normalisent les chemins et ciblent le premier champ invalide.

5.3 Limites actuelles

- L'application est destinée à un usage interne et ne contient pas encore de portail public de réservation.
- Les reçus et récapitulatifs de paiement ne constituent pas une facturation fiscale complète.
- Le projet n'intègre pas de passerelle bancaire réelle, de paiement en ligne ou de connexion à un logiciel comptable.
- La notification par courrier électronique ou SMS n'est pas incluse.
- Le déploiement de production, la supervision et les sauvegardes automatisées restent à mettre en place selon l'environnement cible.

5.4 Perspectives

- Déployer la solution sur une infrastructure de production avec HTTPS, sauvegardes et supervision.
- Ajouter un portail public de réservation connecté aux mêmes règles de disponibilité.
- Intégrer les notifications email/SMS pour les confirmations, acomptes et réclamations.
- Mettre en place une journalisation générale des actions sensibles.
- Développer une facturation complète et une intégration comptable.
- Ajouter des tableaux de bord analytiques avancés et des prévisions d'occupation.
- Renforcer les tests automatisés frontend et les tests end-to-end.

Conclusion générale

Ce projet de fin de formation a permis de concevoir et de développer un système web de gestion hôtelière complet, depuis l'analyse des besoins jusqu'à l'implémentation et à la validation fonctionnelle. La solution centralise les opérations essentielles dans une interface interne sécurisée et responsive.

Le travail réalisé a dépassé un simple ensemble de pages CRUD. Il a nécessité l'application de règles métier liées à la disponibilité des chambres, à la tarification, à l'occupation, aux paiements, aux réclamations, aux comptes utilisateurs et à la traçabilité. La séparation entre React/Vite et l'API Laravel 11 a favorisé une architecture claire et maintenable.

La mise en place de Laravel Sanctum, des rôles, du statut actif, de la révocation des jetons et des protections du dernier administrateur a renforcé la sécurité. Les composants communs, la validation inline, les exports et les documents de paiement ont amélioré l'expérience utilisateur. Enfin, les tests backend et la compilation de production ont fourni une base de confiance pour la stabilité de la solution.

Cette réalisation m'a permis de consolider mes compétences en développement full stack, modélisation, bases de données, sécurité, tests, gestion de versions et résolution de problèmes. Elle constitue une base évolutive qui pourra être enrichie par un déploiement de production, un portail public, des notifications et des intégrations externes.

Bibliographie et webographie

1. Documentation officielle Laravel 11 — architecture, Eloquent, validation, middleware et tests.
2. Documentation officielle Laravel Sanctum — authentification des API et personal access tokens.
3. Documentation React — composants, hooks et gestion de l'état.
4. Documentation Vite — environnement de développement et build de production.
5. Documentation MySQL — conception relationnelle, contraintes et transactions.
6. Documentation Axios — requêtes HTTP et intercepteurs.
7. Documentation Bootstrap et React-Bootstrap — composants responsive et formulaires.
8. Documentation jsPDF et AutoTable — génération de documents PDF.
9. Object Management Group — Unified Modeling Language (UML).

Annexes

Annexe A — Principaux endpoints

Domaine	Endpoints / opérations
Authentification	POST /api/login ; POST /api/logout ; GET /api/user
Profil	GET/PUT /api/profile ; PUT /api/profile/password ; POST/DELETE /api/profile/photo

Utilisateurs	GET/POST /api/users ; PUT /api/users/{user} ; PATCH status et reset-password
Réservations	Liste, création, consultation, modification, suppression, disponibilité et calcul tarifaire
Paiements	Création, annulation et consultation de l'historique
Modules métier	Clients, chambres, tarifs, réclamations, équipements et statistiques

Annexe B — Captures et diagrammes à fournir

- Captures finales des écrans listés dans le chapitre 4.
- Diagramme de cas d'utilisation Administrateur / Employé.
- Diagramme de classes ou de domaine basé sur les modèles Laravel.
- Schéma relationnel lisible de la base de données.
- Diagrammes de séquence pour réservation, paiement et réclamation.
- Résultat final de la commande de tests backend et du build frontend.

Statut de cette version

Cette version constitue la version scolaire finale du projet. Les emplacements marqués dans le chapitre fonctionnel doivent être remplacés par les captures d'écran définitives avant la remise.